First Steps with Your QNX System

Lecture 2

Overview

Learning Objectives

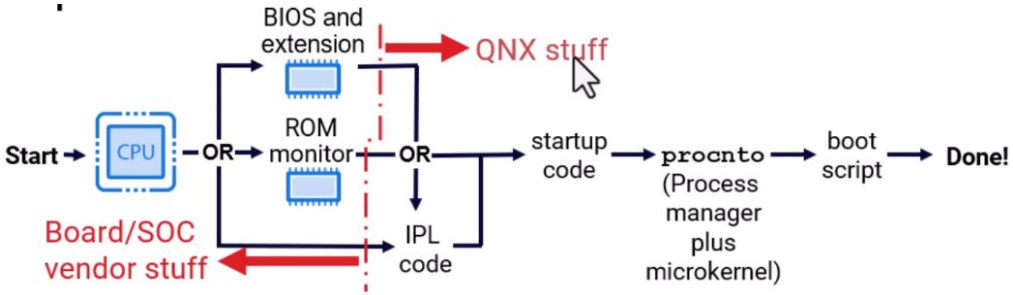
- Understand the QNX boot sequence and image structure
- Navigate the QNX shell and filesystem effectively
- File Ownership and Permissions
- Connect to your Raspberry Pi via WiFi and SSH
- Write, compile, and run your first QNX program
- Master essential QNX development workflow

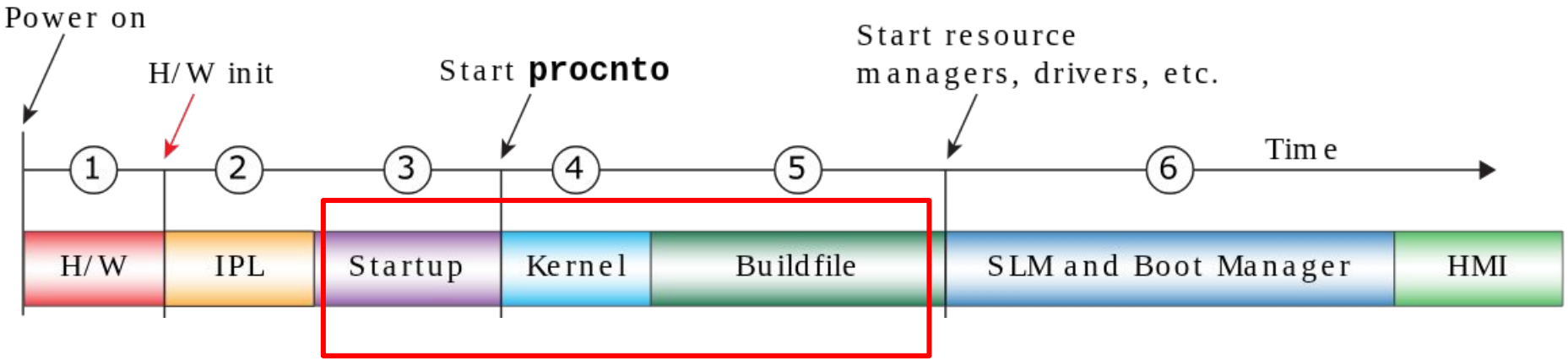
Topic 0 - QNX Booting Sequence

From Power-On to Full OS

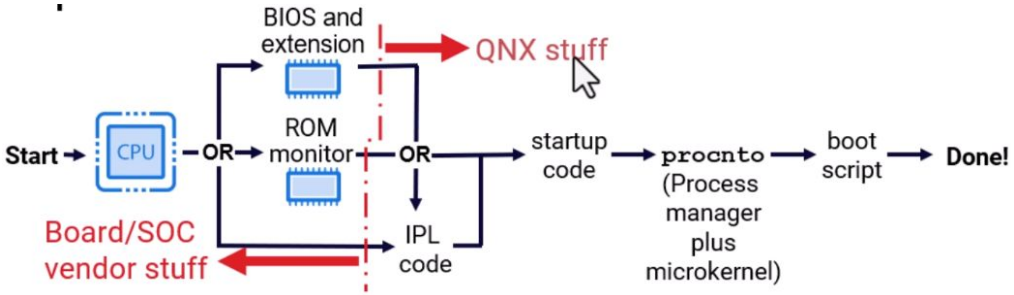
High Level Boot Sequence Overview

A useful way to think about the boot process and its components is that each step provides greater abstraction from the hardware. The IPL is specific to an architecture, board, and sometimes even board revision. The startup code is specific to an architecture and board. And, generally speaking, an OS kernel can work with different boards that share an architecture.





High Level Boot Sequence Overview



Step1: Before QNX — Provided by the board/SOC vendor

- **ROM Monitor** (like u-Boot, RedBoot, Bare-metal loader)
- **BIOS / UEFI** (in x86 systems)
- **On-chip ROM boot** (common in ARM)

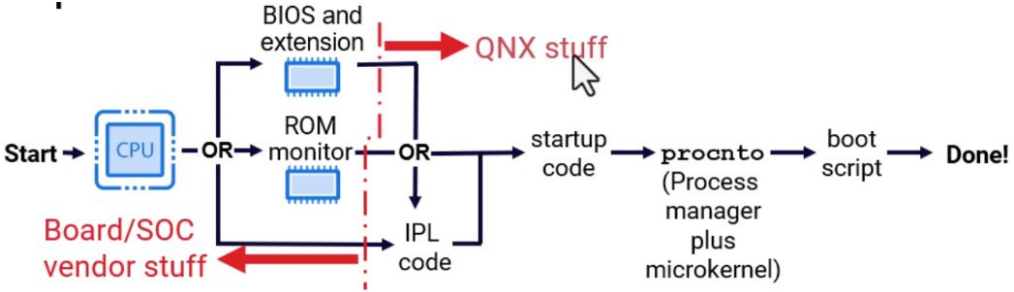
The board hardware and firmware perform checks and initialization tasks, then hand control over to the software located at a predetermined location.
Their job is to: load and execute the **IPL** (Initial Program Loader)

These components are *not* part of QNX.

High Level Boot Sequence Overview

QNX Boot Components

Step 2: IPL (Initial Program Loader)



The **IPL is the first QNX-specific code** that runs.

After finishing basic hardware preparation, it loads and locates the **IFS(Image File System)**

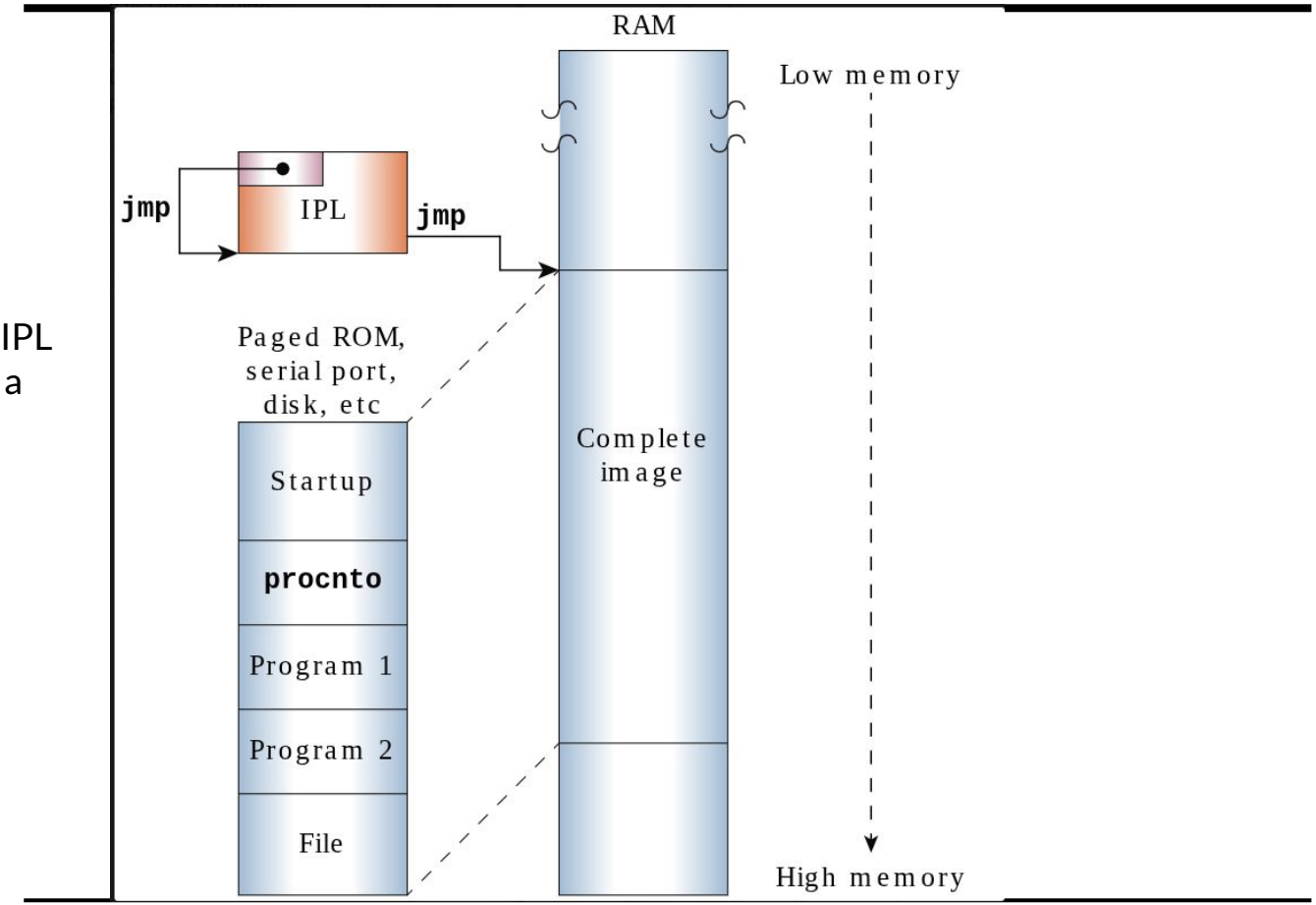
- Complete hardware initializations, if needed.
- Set up a stack for the C-language environment.
- Find and validate the OS image.
- Load the OS image into memory RAM.
- Jump to the startup code in the OS image.

The IPL moves the IFS from its storage location (like an SD card or Flash) into the RAM so that the startup program and QNX kernel can begin running

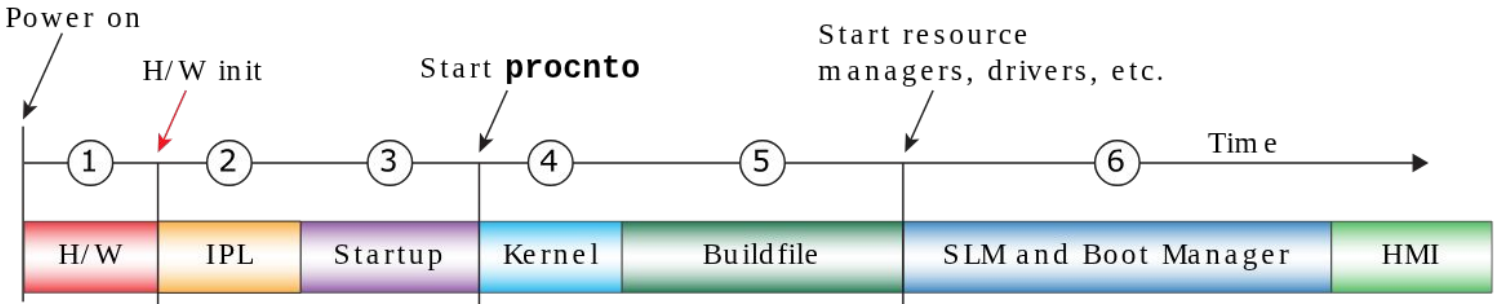
- Warm-start IPL
If there's a BIOS or ROM monitor already installed at the reset vector, then your IPL code is simply an extension to the BIOS or ROM monitor.
- Cold-start IPL
The system doesn't have (or doesn't use) a BIOS or ROM monitor program. The IPL must be located at the reset vector

QNX

Figure showing how a QNX IPL loads the entire image from a non-linearly mapped device into RAM.



High Level Boot Sequence Overview



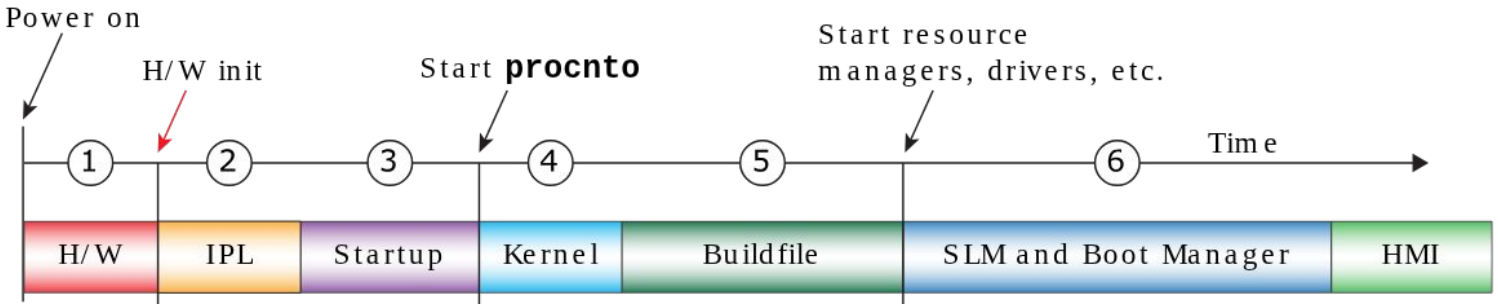
QNX Boot Components

Step 3 : **Startup (Startup Code) : This module takes over after the first-stage bootloader (BIOS, UEFI, U-boot, or IPL) has finished executing. The startup code executes after it has been copied from the boot device into the system's DRAM.**

The startup code's main tasks are, in sequence:

- The startup program configures the system, including timers, interrupt controllers, and cache controllers
- When it has completed the configuration, this program transfers control to the OS kernel.

High Level Boot Sequence Overview



QNX Boot Components

Step 4 : The Kernel : is included in the **procnto** module, which combines the kernel and process manager.

- **This module sets up the kernel, then runs the instructions specified in a script in the buildfile used to generate the OS image. This script contains startup instructions for drivers and other processes, and any additional commands for running other required components.**

High Level Boot Sequence Overview

QNX Boot Components

Step 5: Boot Script(sysinit): is a text file (script) that runs automatically right after the QNX kernel (procnto) finishes starting up.

Its main job is to launch all the necessary services and programs after boot.

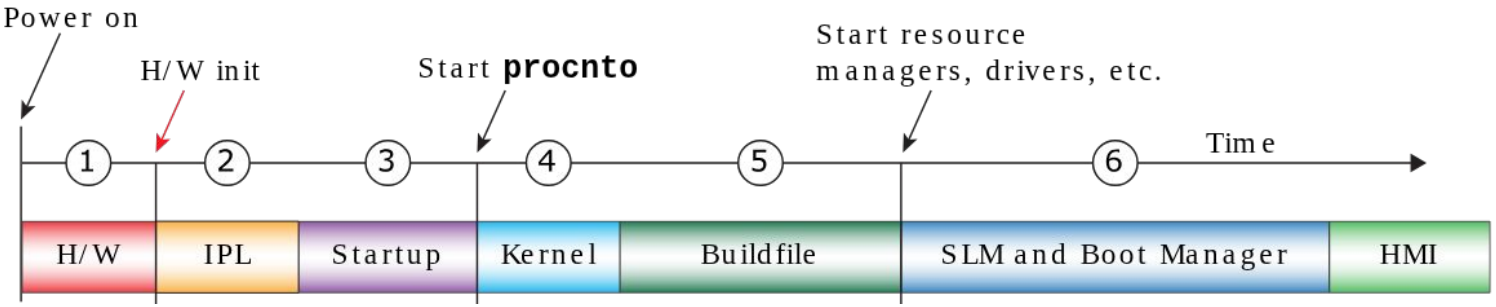
The boot script is where you add:

- the files and commands to include in the image
- the startup order for the executables
- the loading options for the files and the executables
- the command-line arguments and environment variables for the executables

a sequence of commands that you want the kernel and process manager to execute when it has completed its own startup

When all of the commands in the buildfile have completed successfully, the QNX system has finished booting.

High Level Boot Sequence Overview



QNX Boot Components

Step 6 : The System Launch and Monitor (SLM): utility is used to launch complex applications consisting of many processes that must be started in a specific order. It allows you to configure application launch sequences without modifying the buildfile

- If a component isn't required for the initial startup, you can take the component binary out of the IFS and manage its launch with the SLM service. This approach reduces the size of the IFS and makes easier to modify your startup sequence.

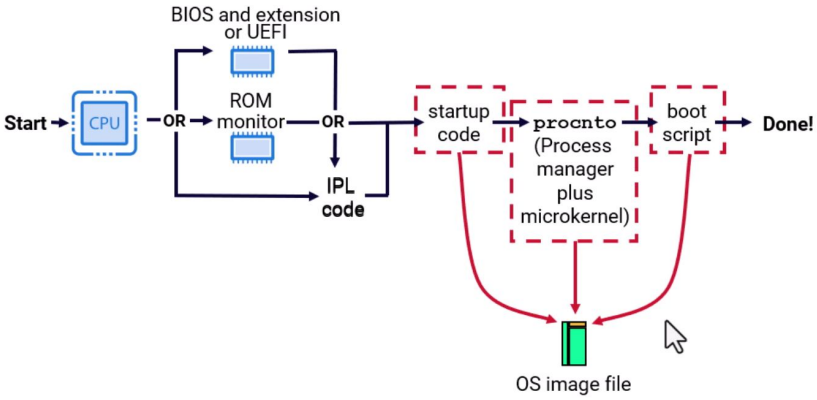
High Level Boot Sequence Overview

QNX Boot Components

IFS (Image Filesystem)

The IFS contains everything QNX needs to start running.

- **Startup code**
- **procnto** → The QNX Neutrino *microkernel* + process manager
- **Boot script** → *Very important*



The time needed to copy the IFS into RAM varies according to the IFS contents (and hence, size). It is usually the longest part of the kernel boot process.

High Level Boot Sequence Overview

QNX Boot Components

Image File Structure

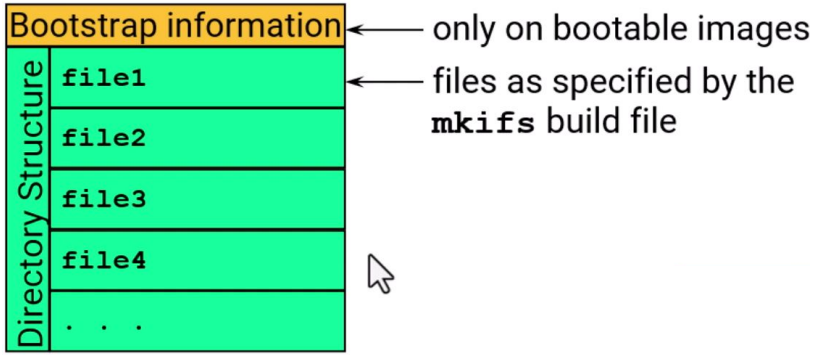
1 — 2

Bootstrap Section

Initial loader information at the front of the file

File System

Contains all files needed at boot time



Design Philosophy

Minimalist: Only essential drivers
Expandable: Load more from flash/SD after boot
Flexible: Can include everything OR just basics
Board-Specific: Tailored to hardware capabilities

Note: Use **dumpifs** to view .ifs file content.

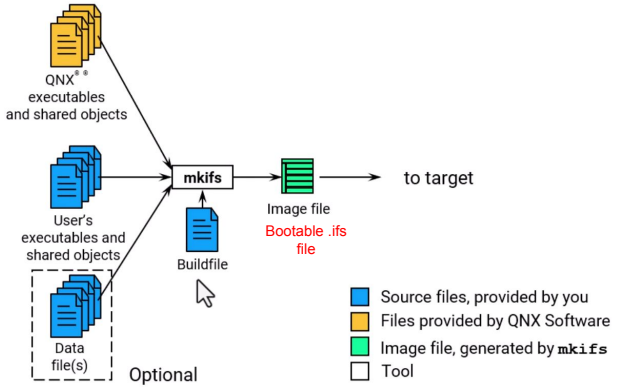
mkifs and the Buildfile

QNX Boot Components

mkifs: Make Image Filesystem

What is a Buildfile?

mkifs uses rules from the buildfile to combine OS and user executables, shared objects and data files to create an OS image (IFS), which can be transferred to the target system and booted.




```
mkifs [-?] [-a suffix] [-l inputline] [-n[n]] [-o directory]
      [-p patchfile] [-r rootdir] [-s section] [-v]
      [buildfile] [directory] [outputfile]
```

mkifs and the Buildfile

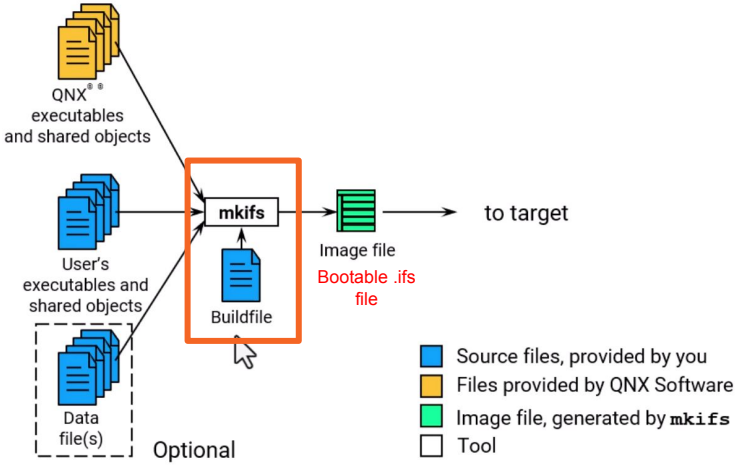
QNX Boot Components

Buildfile

What is a Buildfile?

- a text file that specifies:

- files/commands that will be included in the image
- loading options for files & executables.
- options for creating the image (e.g. compression)
- for bootable images, a boot script that says what to run after OS initialization



What appears in /proc/boot

Once booted, everything you put in the image file appears in /proc/boot by default. This is a read-only, in-memory filesystem. It is not on any disk — it is the image itself, mapped into RAM. You can redirect files to appear at other paths (like /etc instead of /proc/boot) using path assignments in the buildfile.

QNX

QNX Boot Components

Buildfile Essential Contents

mkifs and the Buildfile

- bootstrap loader:
  - startup-boardname
- and operating system:
  - procnto-smp-instr
- boot script
- executables and shared libraries
  - executables aren't strictly required, but then the system wouldn't actually ***do*** anything without them!
  - but, in most cases, to run any executable, you need at least ***libc.so, libgcc_s.so, and ldqnx-64.so***
  - and most QNX supplied services also require ***libsecpol.so***

target PROCESSOR

```
#     This is "hello.build"
[virtual=x86_64,multiboot] .bootstrap = {
    startup-x86
    PATH=/proc/boot procnto-smp-instr
}
[+script] .script = {
    procmgr_symlink /proc/boot/ldqnx-64.so.2 /usr/lib/ldqnx-64.so.2
    devc-ser8250 -e -b115200 &
    reopen /dev/ser1
    hello
}
libc.so
libgcc_s.so.1
ldqnx-64.so.2
libsecpol.so
devc-ser8250
hello
```

bootstrap file

startup script

shared libraries

executables

mkifs and the Buildfile

The general format for a buildfile is:

[attribute] filename contents
[attribute] filename contents

- Can include blank lines and comments as well (comments begin with the pound sign, "#")
- All components are optional
  - cannot have contents without a filename

mkifs and the Buildfile

QNX Boot Components

Buildfile syntax (Attributes)

Instruction tag you attach to a file or section to change how ***mkifs*** handles it. Without any attribute, ***mkifs*** just copies the file as-is into the image. With an attribute, you can change the file's permissions, owner, where it appears, how it is treated, and much more.

Two types of attributes

Type 1 — Boolean attribute (on/off switch): A boolean attribute is either turned ON or OFF. You control it with a + (plus) for ON and a - (minus) for OFF. You do not give it a value — it is simply enabled or disabled.

 # Syntax:

```
+attributename : turns it ON
-attributename : turns it OFF
```

Examples:

```
+script: marks this block as the boot script (ON)
+session: create a new POSIX session for this program (ON)
+optional: this file is optional, do not fail if not found(ON)
-optional : this file is NOT optional, fail if not found (OFF)
```

Type 2 — Value attribute (name = value): A value attribute takes an actual value that you assign to it. You write the attribute name, an equals sign, then the value **"without spaces"**. Used for things like permissions, user IDs, group IDs, and file types.

 # Syntax:

```
attributename=value
```

Examples:

```
uid=0 ← set file owner to user ID 0 (root)
gid=0 ← set file group to group ID 0
perms=0755 ← set file permissions to rwxr-xr-x
type=link ← this entry is a symbolic link, not a file
type=dir ← create this as an empty directory
autoso=list ← automatically include all .so dependencies
```

Combining multiple attributes on one line

You can put more than one attribute inside one pair of square brackets, separated by spaces or commas. They all apply together to whate*ver comes after them.

```
# Multiple attributes on one file:  [+optional uid=0 perms=0755] mydriver
```

mkifs and the Buildfile

QNX Boot Components

Buildfile syntax (Attributes)

Scope — per-file vs global

The most important rule about attributes — scope

An attribute can apply to either a single file or all subsequent files. This is called its scope. The position of the attribute in the line controls the scope.

Scope 1 — Per-file (inline, before the filename)

Put the attribute in square brackets directly in front of the filename, on the same line. It applies ONLY to that one file. The next file is not affected at all.

Example :

```
[uid=0] mydriver ← uid=0 applies ONLY to mydriver
anotherdriver ← NOT affected — no uid set
[perms=0755] myscript ← perms=0755 applies ONLY to myscript
yetanother ← NOT affected
```

Scope 2 — Global (attribute on its own line)

Put the attribute on a line by itself — no filename after it. It applies to ALL files listed after it, until you change it or turn it off. Think of it as setting a default that sticks.

Example :

```
+optional ← now ALL following files are optional
fileA ← optional
fileB ← optional
fileC ← optional
-optional ← turn it OFF — back to required
fileD ← NOT optional (will fail if missing)
fileE ← NOT optional
```

QNX Boot Components

Buildfile syntax (Attributes)

mkifs and the Buildfile

+script / -script Boolean (+/-) Marks a block as boot script	**+optional / -optional** Boolean (+/-) File may be missing — do not fail
+session / -session Boolean (+/-) Start a new POSIX session	**+compress / -compress** Boolean (+/-) Compress the IFS image, At boot time the startup code automatically decompresses it before procnto runs.
uid=N Value (name=val) Set the file owner (user ID), who can read/write/execute	**gid=N** Value (name=val) Set the file group (group ID)
perms=OCTAL Value (name=val) Set Unix file permissions, bits of the file in the image - same as **chmod** on Linux.	**type=link** Value (name=val) `[type=link] linkname=targetfile` Instead of copying a file, creates a symbolic link in the image.
type=dir Value (name=val) Creates an empty directory in the image at the specified path.	**autoso=list** Value (name=val) Auto-include all .so dependencies
virtual=ARCH,TYPE Value (name=val) Used only in the bootstrap section. Define CPU arch and image type **ARCH:** x86_64 (Intel/AMD 64-bit), aarch64le (ARM 64-bit, e.g. Pi 4), armle-v7 (ARM 32-bit). **TYPE** image format: bios (for PC BIOS boot, VMware, VirtualBox), raw (for embedded boards, Raspberry Pi, automotive SoCs).	**search=PATH** Value (name=val) By default mkifs searches the QNX SDP paths. [search=] overrides that for specific files.

mkifs and the Buildfile

QNX Boot Components

Buildfile syntax (Adding Files)

devc-ser8250
/etc/hosts = /project_files/target/etc/hosts

To add files to your image, list them in your buildfile :

- find **devc-ser8250** and put it in **/proc/boot**
- explicitly supply path for target (/etc/hosts) and host (/project_files/...)

Or a file can be given in line:

- readme = {
  This is a handy way to get a file into the image without actually having a file. The file, readme, will be accessible as /proc/boot/readme.
  }

mkifs and the Buildfile

QNX Boot Components

Buildfile syntax (Scripts)

Using the [+script] attribute, a file is treated as a script:

- It will be executed after the process manager has completed its startup.
- Multiple scripts will be concatenated into one and be interpreted in the order given.
- There are modifiers that can be placed before commands to run:
  - Example: **[pri=27f] esh**

Built-in commands — procnto handles these directly These are commands that procnto just knows how to do. mkifs also knows they are builtins and will NOT try to find executable files for them.	
display_msg	**Builtin** Prints text to the debug device (serial port or console) at boot time. Useful for debugging the boot sequence — like a printf for the boot script.
procmgr_symlink	**Builtin** Creates a symbolic link in the path space.
reopen	**Builtin** Sets standard input (fd 0), standard output (fd 1), and standard error (fd 2) to a device path. Any program launched after this inherits those file descriptors. Also suspend the interpretation of the script file for the specified amount of time until a stat() on the specified device path succeedsThis is how you attach a shell to a serial port.
waitfor	**Builtin** Pauses the boot script until a path appears in the QNX path space. Essential for synchronisation — e.g. wait for /dev/ser1 before launching a program that needs it. Prevents race conditions at boot.

QNX

QNX Boot Components

Buildfile syntax (Scripts)

mkifs and the Buildfile

- there are no testing, branching, or looping constructs
  - more complicated initialization should be done through scripts or programs

```
[+script] .script = {
    ...
    ksh /proc/boot/startup.sh
}
…

# Your startup.sh can have full logic:
startup.sh{
# #!/bin/ksh
 if [ -f /data/config ]; then
    source /data/config
 fi
}
```

mkifs and the Buildfile

QNX Boot Components

Where files are found

- To find files, mkifs, looks in:

```
${QNX_TARGET}/${PROCESSOR}/bin,
      ../usr/bin, ../sbin, ../usr/sbin
${QNX_TARGET}/${PROCESSOR}/boot/sys
${QNX_TARGET}/${PROCESSOR}/lib,
      ../usr/lib
${QNX_TARGET}/${PROCESSOR}/lib/dll
```

Binaries (esh, ls, etc)
OSes (procnto, etc)

Libraries and shared objects
Shared objects

- The above can be overridden:
  - using the **MKIFS_PATH** environment variable:
    MKIFS_PATH=/usr/nto/x86/bin:
    /usr/nto/x86/sys: /usr/nto/x86/dll:
    /usr/nto/x86/lib: /project/bin
  - most BSPs modify this
    - using the **search** attribute for a particular file:
      [search={MKIFS_PATH}: /project/bin] myexec

mkifs and the Buildfile

Buildfile format

What is a Buildfile?

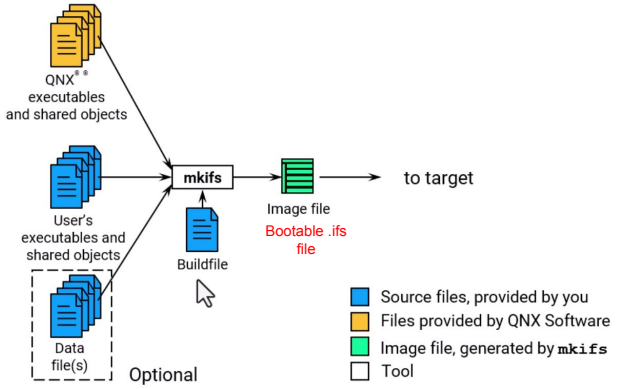
mkifs uses rules from the buildfile to combine OS and user executables, shared objects and data files to create an OS image (IFS), which can be transferred to the target system and booted.

Text Configuration: Created with any text editor

File List: Specifies files to include in image

Command Script: Contains commands for boot script

Essential Component: For bootable images, you *must* include a boot script(startup) that specifies what should run **after procnto is ready to go.**



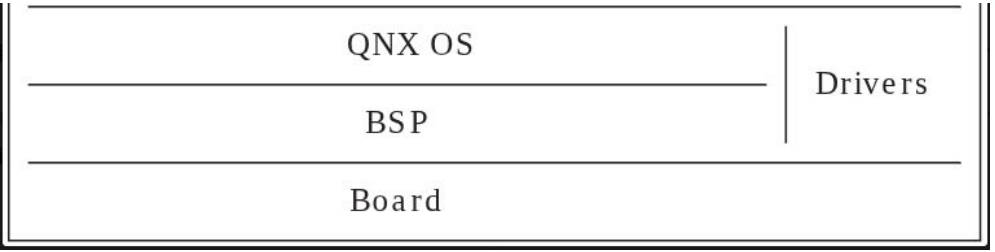

```
mkifs shell.bld shell.ifs
```

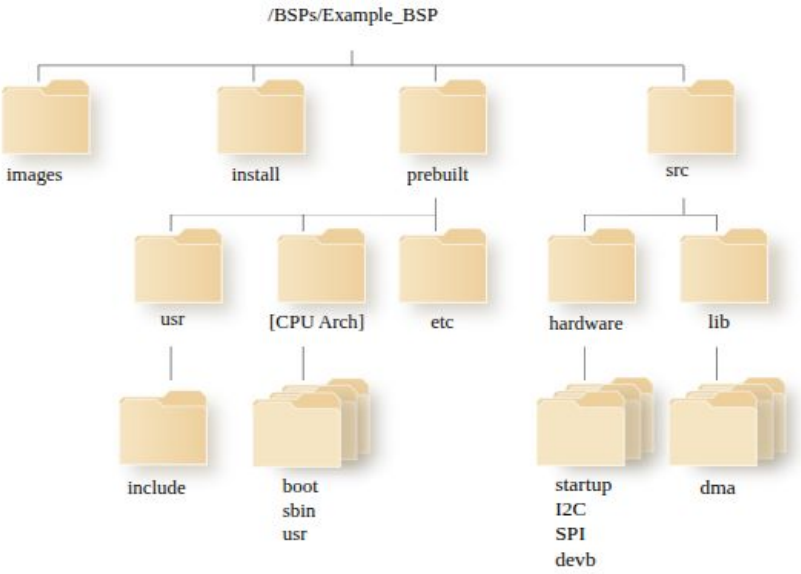
This command tells mkifs to use the buildfile **shell.bld** to create the image file **shell.ifs**.

What appears in /proc/boot

Once booted, everything you put in the image file appears in /proc/boot by default. This is a read-only, in-memory filesystem. It is not on any disk — it is the image itself, mapped into RAM. You can redirect files to appear at other paths (like /etc instead of /proc/boot) using path assignments in the buildfile.

BSP structure and contents





Link

Download and set up the BSP

1- Download Board Support Packages (BSPs) using the QNX Software Center.
- Add Installation -> Search(BSP - RPI5) -> Install -> Finish

2- Navigate to (qnx800) Dir -> TO (bsp) Dir
-> Extract(**BSP_raspberrypi-bcm2712-rpi5_be-800_*build_ID*.zip**)

3- Navigate to (BSP_raspberrypi-bcm2712-rpi5_be-800_build_ID.zip) Dir -> Images Dir
-> Open in Terminal -> Run "`make clean`" then , Navigate to main Dir "`cd ..`"
-> Run "`make`"

4- **Prepare a bootable microSD card**
- Insert your SD-card, and detect it's Device Name (eg.., **sda1** or **mmcblk0p1**; what you see may differ on your computer), and mount pioint using "`mount`", or "`lsblk`" commands -> If the device is mounted, unmount it "`umount /dev/sda1`"
- Using administrator privileges, run the following commands :
  - Run "`sudo cfdisk /dev/sda`" -> "Delete" all you current partitions

  -> Create "New" Partitions:
  1- Size = 200M || Primary || Type = "FAT16" || Bootable => Then "Write"
  2- Size = Rest || Extended || Type = "Linux" => Then "Write"

  -> Then "Quit"
  - Run "`sudo mkfs.vfat -F 16 -n boot /dev/sdb1`" to make file system for Partition 1

Download and set up the BSP

- Copy the necessary binaries and IFS to microSD card :
  - The Raspberry Pi 5 board needs certain firmware files to boot, such as:
    - "`config.txt, bcm2712-rpi-5-b.dtb, bcm2712d0-rpi-5-b.dtb, overlays/`"
  - Prepare boot config file :

```
[rpi5]
 enable_uart=1
 force_turbo=1
 kernel=ifs-rpi5.bin
```

  - Move "ifs-rpi5.bin" from "*$BSP_ROOT_DIR*/images" to the SDCard Partion1 mount point

5- Connect the hardware (Raspberry Pi Debug Probe, Ethernet cable, and Raspberry Pi 5 Power Supply)

6- Boot the Image

Download and set up the Quick Start Image

Topic 1: Exploring The System

The Shell (ksh) Environment

The default shell on QNX systems is a variant of the **Korn Shell**, called **ksh**

Why ksh?

- Lightweight, POSIX-compliant shell
- Small memory footprint (important for embedded)
- Familiar syntax for Linux/Bash users
- Built-in job control and command history
- Scripting capabilities similar to Bash

- Linux systems that use Busybox shell utility, QNX simplifies this by using **Toybox** .
- The system uses symbolic links from various utility names to toybox, which allows the user to use standard shell commands.

run "**echo $0**" to check the shell name

The Shell (ksh) Environment

Unix	QNX OS	See also:
adduser	passwd	Managing User Accounts
at	cron	
dmesg	slogger2, slog2info	
fsck	chkqnx6fs, chkdosfs	
ifconfig eth0	ifconfig em0	TCP/IP Networking
man	use	Using the Command Line
pg	less, more	Using the Command Line

Basic commands

The Shell (ksh) Environment

Filesystem Architecture

- A file system organizes and stores data in a structured way, making it possible for users and programs to access files easily.
- A filesystem in QNX isn't part of the kernel itself. It runs as a process that communicates with applications via the POSIX API (open, read, write, close, etc.).
- In QNX, the filesystem is not a disk — it is a live address book assembled at runtime by independent processes.

The Three Pillars

- **There is no single root disk.** The root filesystem **/** is not mounted from a storage partition. It exists as a virtual address book in RAM, maintained by the process manager (procnto).
- **Every path is registered by a running process.** A path like /dev/ser1 does not exist until the serial port driver process starts and registers that address. Kill the driver, the path disappears.
- **The path space is virtual.** It only exists in memory while the system is running. There is no file on any disk that holds the directory tree. It is assembled live at boot time.

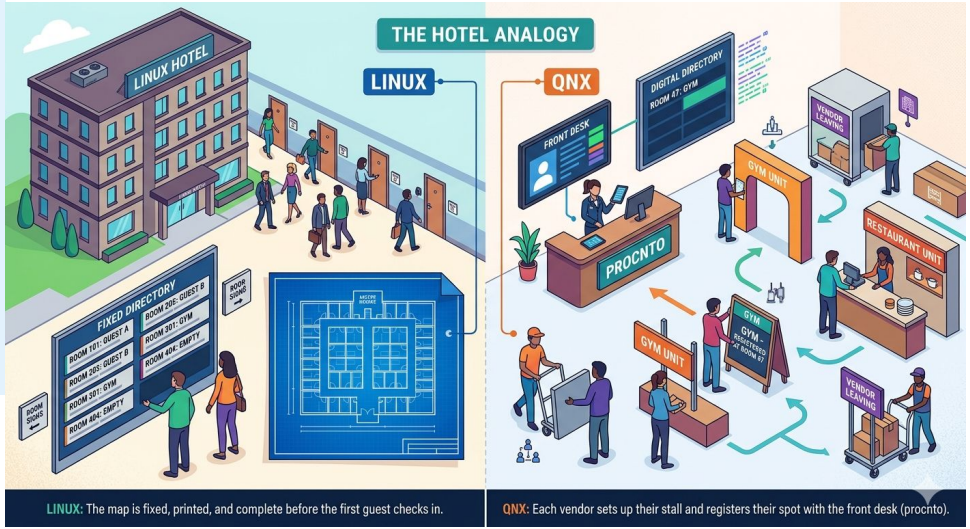
Path Space / : The collection of all paths from all filesystems is called the *path space*, managed by the **path manager** in the QNX kernel.

The Hotel Analogy

The easiest way to understand QNX is through a simple analogy. Think of the filesystem as a hotel directory.

- **Linux** is a hotel where the manager built all the rooms before guests arrived. The map is fixed, printed, and complete before the first guest (application) checks in. Every room exists whether it is occupied or not.
- **QNX** is a hotel where each vendor shows up, sets up their stall, and registers their spot with the front desk (procnto). The gym only appears in the directory because the gym company showed up and announced: "We are at room 47." If the vendor leaves, room 47 disappears from the directory. The front desk does not own any rooms — it just keeps the address book.

This is exactly what happens with `/dev/ser1` in QNX. The serial port driver process starts up, sets up its "stall," and registers the address `/dev/ser1` with the process manager. Your app asks for `/dev/ser1`, the process manager checks its address book, and routes you to that driver process. Kill the driver — the path vanishes.



Resource Managers

Every path in the QNX filesystem is owned by a resource manager. Understanding resource managers is the key to understanding everything else about QNX.

What is a Resource Manager?

A resource manager is a regular user-space process that does three things:

1. It starts up like any other program
2. It claims ownership of a path (for example, /dev/ser1)
3. It registers that path with procnto via a system call
4. It then answers any file operation that arrives at its path — open, read, write, close, seek

For example, a serial port driver starts up and says: *"I am responsible for anything under `/dev/ser`."* From that moment, any app that opens `/dev/ser1` gets routed to that driver. The driver itself decides what files appear under its claimed path — that's what "serviced by the resource manager" means. It services (handles, responds to) any request that comes to that path.

procnto — The Process Manager

procnto is the QNX microkernel combined with the process manager. Its role in the filesystem is minimal but critical:

1. It maintains a lookup table of all registered paths in RAM
2. When an application opens a path, it checks the table and routes the request to the correct process
3. It directly owns /dev and /proc — the two core virtual directories
4. It does not own any files, binaries, or configuration data

Key insight:

procnto is like a telephone exchange operator. It connects the caller to the right party — but it is not the party itself. The moment procnto starts, it creates an empty path space. Every subsequent path that appears was registered by another process that started after procnto.

QNX

“QNX doesn't prescribe a specific filesystem layout — the designer is free to structure files to fit the goals of the system”

The filesystem is virtual and built at runtime by processes, you as the system builder decide what it looks like. On a minimal car ECU running QNX, you might have only five paths total. On a full QNX desktop you might have hundreds. Nobody forces you to have `/home` or `/usr` — those only appear if you choose to include the processes and files that create them.

Compare this to Linux, where if you remove `/etc/passwd` or `/lib/ld-linux.so` the system breaks — because the kernel and standard tools hardcode those locations. QNX only has two mandatory paths: `/usr/lib/ldqnx.so.2` (the dynamic linker) and `/.boot` on x86. Everything else is your choice.

You need a way to see what's currently registered.

`/proc/mount` is — a live snapshot of the process manager's address book at any given moment.

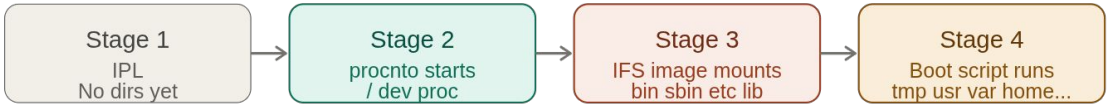
- This is the **live address book** of your running QNX system. Every entry you see is either a registered resource manager path or a directory in the path space.

```
# ls /proc/mount
0,1,1,1,4294967295  0,1,1,19,8          0,163853,1,8,11  boot  system
0,1,1,12,5          0,1,1,2,4294967295  0,258064,1,0,6   data  usr
0,1,1,14,13         0,1,1,21,0          0,65544,3,0,11   dev
0,1,1,18,11         0,143370,1,0,4      bin              proc
```

```
ND , PID , CHID , HANDLE , FLAGS
```

Field	Meaning
`ND` (Node Descriptor)	Which machine in the network. `0` always means **this local machine**
`PID` (Process ID)	The process ID of the resource manager that owns this path
`CHID` (Channel ID)	The communication channel inside that process that handles requests
`HANDLE`	A specific sub-handle within that resource manager
`FLAGS`	Behaviour flags for this mountpoint

The Boot Sequence — How the Filesystem is Built



Directory	Created by	When	Real or virtual?
/	procnto	Instant — first thing to exist	Virtual
/dev	procnto	Stage 2 — procnto owns it	Virtual
/proc	procnto	Stage 2 — procnto owns it	Virtual
/bin	IFS image (mkifs)	Stage 3 — IFS mounts at /	ROM/Flash
/sbin	IFS image (mkifs)	Stage 3 — IFS mounts at /	ROM/Flash
/etc	IFS image (mkifs)	Stage 3 — IFS mounts at /	ROM/Flash
/lib	IFS image (mkifs)	Stage 3 — IFS mounts at /	ROM/Flash
/boot	IFS image (mkifs)	Stage 3 — IFS mounts at /	ROM/Flash
/system	IFS image (mkifs)	Stage 3 — IFS mounts at /	ROM/Flash
/tmp	Boot script / procnto	Stage 4 — alias to /dev/shmem	Virtual (RAM)
/usr	Filesystem driver	Stage 4 — flash fs mounts it	Real (flash/disk)
/var	Filesystem driver	Stage 4 — flash fs mounts it	Real (flash/disk)
/home	Filesystem driver	Stage 4 — flash fs mounts it	Real (flash/disk)
/data	Filesystem driver	Stage 4 — flash fs mounts it	Real (flash/disk)
/root	Filesystem driver	Stage 4 — flash fs mounts it	Real (flash/disk)
/usb	USB driver (devb-umass)	Stage 4 — only when USB plugged	Real (USB device)

procnto (virtual) IFS boot image Boot script / driver

The Boot Sequence — How the Filesystem is Built

Step 1 — Power on: IPL runs

The Initial Program Loader (IPL) wakes up in flash memory. It sets up RAM, configures clocks and memory controllers, finds the IFS binary on the SD card or flash chip, copies it into RAM, and jumps to it. At this point, **the filesystem does not exist at all**.

Step 2 — Startup program runs

The startup program (baked into the IFS) reads the system hardware and builds a system page in memory — a data structure describing the CPU, memory map, interrupt lines, and cache sizes. It then hands control to **procnto**.

Step 3 — procnto starts: filesystem is born

This is the most important moment. When procnto initialises:

- It creates an empty virtual path space /
- It immediately registers /dev and /proc as its own virtual directories
- It mounts the IFS — making /bin, /lib, /etc, /usr, /sbin and other baked-in directories visible
- /proc/boot opens as a window into the IFS contents

The Boot Sequence — How the Filesystem is Built

Step 4 — Boot script launches drivers

The boot script inside the IFS runs and launches driver processes one by one. Each process starts, claims its path, and calls **resmgr_attach()** to register with procnto. For example:

- **Serial driver starts** → registers /dev/ser1, /dev/ser2
- **Shared memory manager starts** → registers /dev/shmem/
- **Network stack starts** → registers /dev/socket/
- **USB stack starts** → registers /dev/usb/

Step 5 — Persistent storage mounts

Filesystem drivers (such as the QNX6 Power-Safe filesystem driver) start and mount real storage partitions. /data and /var get their contents from the SD card partition. The /tmp alias to /dev/shmem is configured.

Step 6 — Filesystem is fully assembled

Running ls / now shows all directories — but each one was created by a different process at a different moment in time. The filesystem looks like a unified whole, but it is actually a collection of independently registered path spaces maintained by many separate processes.

Directory Explore - QNX PI

/proc (Stage 1)

Property	Detail
Made by	procnto (process manager) itself
Made when	The instant procnto started — first moment of OS
Lives on	Nowhere on disk — entirely virtual, in procnto memory
Storage type	Virtual (RAM)

/proc is the first core virtual directory owned by procnto. It is more powerful than its Linux equivalent:

- **/proc/<PID>/** — one directory per running process. You can read and write into a process's memory space through these paths. This is how debuggers and system monitors work on QNX.
- **/proc/boot/** — a live window into the IFS boot image. Shows you every file that was baked into flash. Unique to QNX — Linux has no equivalent.
- **/proc/mount** — the live address book. Every entry shows which process has registered which path. This is the master registry that procnto consults for every file open operation.

Directory Explore - QNX PI

/dev (Stage 1)

Property	Detail
Made by	procnto (process manager) itself
Made when	The instant procnto started — first moment of OS
Lives on	Nowhere on disk — 100% virtual, in procnto RAM
Storage type	Virtual (RAM)

/dev is the second core virtual directories created directly by procnto. It starts empty and fills up as driver processes register their paths. Every entry under /dev is a doorway to a running process — not a file on any disk. The table below shows common /dev entries and which process owns them:

Directory Explore - QNX PI

`bin`, `sbin`, `lib`, `usr`, `etc`, `root`, `home`, `boot` — Born at build time, live on the SD card

Property	Detail
Made by	BSP developer
Made when	Build time on host PC
Lives on	Inside the IFS binary in flash memory
Storage type	Read-only (IFS)

/home: On minimal embedded QNX systems like the Pi, /home is typically empty — there are no interactive users. It exists as a placeholder because some applications check for it.

/etc: Because /etc is in the IFS, it is read-only. This means configuration changes require rebuilding the image. For systems that need runtime-editable configuration, the BSP developer should arrange for /etc to be overlaid by a writable persistent storage mount.

/boot : The directory holds boot-related files and scripts, On ARM systems like the Pi it holds boot configuration and related files.

/root: The home directory for the root user account. Its path is referenced in /etc/passwd. Because it is in the IFS, any files created here during a session are lost on reboot

/lib: contains the essential shared libraries that every executable on the system needs to run. These are .so (shared object) files that get loaded at runtime by the dynamic linker.

/bin: directory contains the essential user-space utilities needed to operate the system programs like ls, chmod, cp, ksh (the KornShell), cat, grep, and other standard commands. Everything here was listed in the buildfile and packaged by mkifs before the Pi ever powered on.

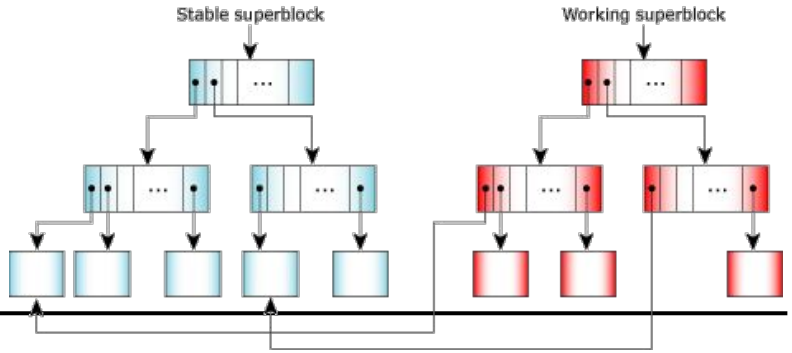
Supported filesystems on QNX OS

What is the Power-Safe Filesystem (QNX6)?

The Power-Safe filesystem, supported by the `fs-qnx6.so` shared object, is a reliable disk filesystem that can withstand power failures without losing or corrupting data.

The Power-Safe filesystem operates by moving the on-disk filesystem state from one stable view to another stable view using copy-on-write (COW) to relocate modified blocks. To finalise this transition, all dirty blocks involved in the new view must be committed to persistent storage, and then a new filesystem superblock/root referencing the relocated blocks is committed. This provides power-safe robustness, because at any point in time either the old version is completely accessible or the new version is completely accessible with no live data being overwritten in between.

When QNX writes data to disk, it never overwrites the old data directly. It writes the new version to a different location first, then atomically switches a pointer to say "the new version is now the valid one." If power cuts out at any moment during this process, you always land on a complete, valid version — either the old one or the new one. You can never end up with a half-written, corrupted state.



Filesystems: /system and /data, Power-Safe Separated Architecture

- This is the most important new design introduced properly in QNX SDP 7.1 and made standard in QNX 8.0. The idea is to split what was previously one big pile of files into three clearly separated zones, each with a different purpose, different trust level, and different storage type.

Zone 1 — `/` (the IFS, baked into flash) : It is read-only, trusted, and contains the absolute minimum needed to get the OS running — the kernel (`procnto`), the startup program, and the essential files that must exist before anything else can run.

Zone 2 — `/system` (a read-only, trusted Power-Safe partition): This is the new zone introduced in QNX 7.1. What lives here is everything that is considered part of the QNX OS itself — OS-level binaries, shared libraries, system services — but too large or too numerous to bake into the IFS image every time. It is mounted read-only and marked as trusted, meaning the OS will execute binaries from here as trusted code

```
root@console:/# mount -f
ifs on / type ifs (rdonly,trusted)
/dev/hd0t179 on /data type qnx6 (rmvable)
/dev/hd0t12 on /boot type dos (fat16) (rmvable)
/dev/hd0t178 on /system type qnx6 (noatime,rmvable)
```

Zone 3 — `/data` (a writable, untrusted Power-Safe partition) : This is the writable partition for everything that changes at runtime — application data, logs, user files, variable state. It is formatted with the Power-Safe filesystem so writes survive power cuts. But critically, it is **not trusted** — binaries placed in `/data` cannot be executed as trusted code.

Download and set up the BSP

7- Make a writable Power Safe QNX6 Partition for on your sd card using “mkqnx6fs”

- mkqnx6fs /dev/sd0t131
- mount -t qnx6 /dev/sd0t131 /home

The FileSystem

- A file system organizes and stores data in a structured way, making it possible for users and programs to access files easily.
- A *filesystem* in QNX isn't part of the kernel itself. It runs as a **process** that communicates with applications via the POSIX API (open, read, write, close, etc.).

Filesystem Classification in QNX

Image: Built into the system image, load into RAM during boot, read-only. /prc/boot
Block: Standard on-disk filesystems (QNX4, DOS, SD-Card). /system, /data , and /boot
Flash: Designed for flash devices (FFS3 for NOR, ETFS for NAND).
Network: Remote access (NFS, CIFS/SMB).
Virtual: Special managers (Package filesystem, Inflator).

Path Space / : The collection of all paths from all filesystems is called the *path space*, managed by the **path manager** in the QNX kernel.

```
qnxuser@qnxpi:~$ df
Filesystem 1K-blocks Used Available Use% Mounted on
ifs 26344 26344 0 100% /
/dev/hd0t17 26212320 1288784 24923536 5% /data/
/dev/hd0t17 2097120 600088 1497032 29% /system/
/dev/hd0t11 2093016 31256 2061760 2% /boot/
/dev/hd0 30408704 30408704 0 100%
/dev/shmem 0 0 0 100% (/dev/shmem)
```

Filesystems: /system and /data, Power-Safe Separated Architecture

- The traditional paths, such as /bin or /usr/lib. The answer is that you can still create a file system on QNX with these paths and mount it at the root directory.
- Filesystem Purpose Typical Contents Permissions **/system** System executables & libraries /system/bin, /system/lib, /system/etc Often **read-only, /data** User data & runtime files /data/home, /data/tmp, /data/var **Read-write**

Benefits of Separation

Security: /system can be mounted read-only with verification/encryption
Reliability: System files protected from accidental modification
Flexibility: /data remains writable for user files and configuration
Maintenance: Easier system updates and recovery

```
# /system directories
/system/bin/ # Executables
/system/lib/ # Libraries
/system/etc/ # Configuration files

# /data directories
/data/home/ # User home directories
/data/tmp/ # Temporary files (also /tmp)
/data/var/ # Runtime data for services
```

Topic 2 - File Ownership and Permissions

Same POSIX model

File Ownership(uid/gid)

What uid and gid are

- Every file has two numbers attached: **a user ID (uid)** and a **group ID (gid)**. These are just integers. The OS does not care about names it only works with numbers.
- The **/etc/passwd** and **/etc/group** files translate numbers into human-readable names.

Permissions are divided into these categories:	
u	Permissions for the user (i.e., the owner).
g	Permissions for the group.
o	Permissions for others (i.e., everyone who isn't in the group).

Each set of permissions includes:	
r	Read permission. For a directory, this is permission to list the directory.
w	Write permission.
x	Execute permission. For a directory, this is permission to search the directory.
s/S	Setuid or setgid
t/T	Sticky bit

User and Groups

- QNX understands Users and Groups
- A user can belong to several groups
- A file can belong to only one user and one group at a time
- The QNX OS recognizes user ID 0 as being privileged, and traditionally an account with uid 0 is called root.
  - This user can do anything on the system; it has what Windows calls administrator's privileges. Unix-style operating systems often call root the superuser.
- Only root can change the ownership of a file

User and Groups

- User information in /etc/passwd
- Password info is in /etc/shadow
- Group information is in /etc/group
- /etc/passwd and /etc/group divide data fields using “:”
- /etc/passwd:
- joeuser:x:1000:1000:Joe User,,,:/home/joeuser:/bin/bash
- /etc/group:
- joeuser:x:1000:

QNX

Permission Bits

Every file has a 10-character permission string. The first character is the file type. The next nine are three groups of three: owner, group, others.

	Regular File	Directory
r`	read the file contents	list directory contents
w	write / modify the file	create/delete files in it
x	execute (runasprogram)	enter the directory (cd)

The permission string — reading `ls -l output`

```
-r-xr-xr-x  1   root     root     68524   2011-12-19 07:18 /usr/bin/top

---------- --- -------  -------  -------- ------------    -------------
    |      |    |        |        |        |                  |
    |      |    |        |        |        |              File Name
    |      |    |        |        |        |
    |      |    |        |        |        +---   Modification Time/Date
    |      |    |        |        |
    |      |    |        |        +-------------   Size (in bytes
    |      |    |        |
    |      |    |        +-----------------------      Group
    |      |    |
    |      |    +-------------------------------      Owner
    |      |
    |      |                                                  count"
    |      |
    +                                                         Permissions

oup
 T                                                            le's owner.
her
 T
le
 T                                                            ar (ordinary) file. A
 in                                                           ad, write, and
cut                                                           the file's group, and
 fi
```

```
d = directory
- = regular file
l = symbolic link
n = Named special
s = Unix domain socket
p = named pipe
c = character special file
b = block special file
```

File permissions

There are two ways to set permissions when using the `chmod` command:

Symbolic mode:

testfile has permissions of `-r--r--r--`

```
                                U   G   O*

$ chmod g+x testfile          ==>-r--r-xr--

$ chmod u+wx testfile      ==>-rwxr-xr--

$ chmod ug-x testfile      ==>-rw--r--r--

U=user, G=group, O=other (world)
```

File permissions

Each group of three bits is represented as a single digit 0–7. This is how you set permissions in chmod and in the buildfile perms= attribute.

```
r = 4 (binary 100)
w = 2 (binary 010)
x = 1 (binary 001)

Add them together for each group:
rwx = 4+2+1 = 7
r-x = 4+0+1 = 5
r-- = 4+0+0 = 4
--- = 0+0+0 = 0

So -rwxr-xr-- = 0754
```

```
4000 = setuid (s in owner execute position)
     When run, the program gets the FILE
OWNER'S privileges
     Example: -rwsr-xr-x = 4755

2000 = setgid (s in group execute position)
     When run, gets the FILE GROUP's
privileges
     Example: -rwxr-sr-x = 2755

1000 = sticky bit (t in others execute
position)
     On directories: only the file OWNER can
delete their files
     Example: drwxrwxrwt = 1777 (used for
/tmp)
```

Utilities to control ownership and permissions:

TO:	USE:
Specify the permissions for a file or directory	chmod
Change the owner (and optionally the group) for a file or directory	chown
Change the group for a file or directory	chgrp

Root on QNX

What root means on QNX — the full definition

root is the user with uid=0. On both Linux and QNX, uid=0 is the superuser. But what uid=0 can actually DO is fundamentally different between the two systems, and this is the most important distinction to understand.

On Linux — root is all-powerful, completely	On QNX — root is much more nuanced (Abilities system)
On Linux, if you are uid=0, you can do literally anything: bypass all file permission checks, load kernel modules, access any process's memory, mount filesystems, change system time, kill any process, open any device. There is no further subdivision. Root is binary — you either have it all or you don't.	QNX introduced an "Abilities" system (also called privileges). Even if you are uid=0, you can only do what your process has been explicitly granted the ability to do. The system breaks root power into fine-grained individual abilities that can be granted or denied to any process independently.

```
# Some QNX abilities — each is an independent grant:
PROCMGR_AID_SPAWN ← can create new processes
PROCMGR_AID_SETUID ← can change uid of processes
PROCMGR_AID_PATHSPACE ← can register paths in path space
PROCMGR_AID_REBOOT ← can reboot the system
PROCMGR_AID_UNTRUSTED_EXEC ← can run untrusted binaries
PROCMGR_AID_MEM_PHYS ← can access physical memory

# A process can be uid=0 but still DENIED specific abilities:
on -A deny,all,inherit,untrusted_exec ksh
# This ksh runs as root but CANNOT execute untrusted binaries
```

Process isolation

Each driver/server is a separate process with its own uid/gid. A compromised driver only has ITS OWN permissions — it cannot affect other driver processes or the kernel.

Root on QNX

The Pathtrust implication — why this matters for security

When pathtrust is enabled (QNX 8.0 security feature), only binaries from trusted filesystems (IFS and /system partition) can be executed — regardless of who is running them. Even root cannot execute untrusted binaries if the process does not have the PROCMGR_AID_UNTRUSTED_EXEC ability.

```
# This WORKS — binary from trusted /proc/boot:
/proc/boot/echo Hello → Hello

# This FAILS — even for root, without untrusted_exec ability:
/data/echo Hello → Operation not permitted

# On Linux: root can run ANY binary from ANYWHERE. Always.
# On QNX: root can be restricted to only trusted binaries.
```

Topic 3 - Booting & Connecting to Raspberry Pi

Multiple ways to access your QNX system

Method 1: Serial Console (Essential for Embedded)

Hardware Connection

RPi GPIO Pins → USB-to-TTL Cable → Host Computer

Pinout: GPIO14 (TXD) → RXD , GPIO15 (RXD) → TXD, GND → GND

```
sudo minicom -b 115200 -D /dev/ttyUSB0
```

Why Serial is Critical:

- Works before networking is initialized, and get **ip "ifconfig"**
- Essential for debugging boot issues
- Low-level access to kernel messages

Method 2: Network Connection (SSH)

Connecting to Personal WiFi

```
bcm0: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> metric 0 mtu 1500
        options=8<VLAN_MTU>
        ether dc:a6:32:bf:29:cb
        inet6 fe80::dea6:32ff:febf:29cb%bcm0 prefixlen 64 scopeid 0x6
        inet 192.168.1.3 netmask 0xffffff00 broadcast 192.168.1.255
        media: IEEE 802.11 Wireless Ethernet autoselect
        status: associated
        nd6 options=1<PERFORMNUD>
```

```
awadin@awadin-dell:~$ ssh qnxuser@192.168.1.3
(qnxuser@192.168.1.3) Password:
Welcome to QNX 8.0!
```

Method 2: Network Connection (SSH)

Connecting through Ethernet

```
bcm0: flags=8843<UP,BROADCAST,RUNNING,SIMPLEX,MULTICAST> metric 0 mtu 1500
        options=8<VLAN_MTU>
        ether dc:a6:32:bf:29:cb
        inet6 fe80::dea6:32ff:febf:29cb%bcm0 prefixlen 64 scopeid 0x6
        inet 192.168.1.3 netmask 0xffffff00 broadcast 192.168.1.255
        media: IEEE 802.11 Wireless Ethernet autoselect
        status: associated
        nd6 options=1<PERFORMNUD>
```

```
awadin@awadin-dell:~$ ssh qnxuser@192.168.1.3
(qnxuser@192.168.1.3) Password:
Welcome to QNX 8.0!
```

Why Serial is Critical:

- Move files through **“scp”**
- **Communication over Network**

Writing Your First QNX C Program

Cross-compilation workflow: Host → Target (Command Line)

1 Write Code on development host

```
touch hello.c
```

2 Cross-Compile using qcc for aarch64

```
source /home/awadin/qnx800/qnxsdp-env.sh
qcc -Vgcc_ntoaarch64le -c hello.c -o hello
file hello
```

3 Transfer executable to Raspberry Pi

```
sudo scp hello.c qnxuser@10.42.0.38:/data/home/qnxuser/
```

4 Execute & Debug on target

```
chmod +x hello_qnx
./hello_qnx
```

Writing Your First QNX C Program

Cross-compilation workflow: Host → Target (Momentics IDE)

1 Connect your target using IP
2 Create New Executable for arch64le
3 Write your Code in source file
4 Build the Project
5 Run the executable

Writing Your First QNX C Program

Cross-compilation workflow: Host → Target (VScode QNX Toolkit)

1 Install QNX Toolkit extension	**4** Create new project
2 Getting started(Open QNX Toolkit Walkthrough): • Setting up the QNX Projects Root, the default path for creating projects, targets, and examples (qnxprojects) • Setting up the SDP path for building your QNX applications (qnx.sdpPath)	**5** Write your Code in source file
3 add your target (using IP)	**6** Run the Project in your target

Lab
Build & Test Your First QNX Program

Hands-on exercise

Lab

Make C Code to Print “Hello World! ”

And Make it run on the boot script

Lab

Edit the buildfile:

- Add missing commands like : vi , mkfsqnx6 , else
- edit the startup; message
- mount the system and data partitions
- …

Explore the QNX World

Additional Resources

- QNX Boot Process
- QNX Neutrino FileSystems
- QNX Neutrino Microkernel - Process and Threads
- QNX 8.0 OS User Guide - Process and Threads
- QNX Meeting Early Boot Requirements (Advanced)
- SYMMETRIC MULTIPROCESSING (SMP) AT A GLANCE (Advanced)

Thank You

Goto the Kahoot quiz